Advanced Computer Networks

Networking Data Structures and Algorithms

Lin Wang

Period 2, Fall 2021

VU VRIJE UNIVERSITEIT AMSTERDAM

Course outline

Warm-up

- Introduction (history, principles)
- Networking basics
- **Networking data structures and algorithms**
- Network transport

Data centers

- Data center networking
- Data center transport

Programmability

- Software defined networking
- Network automation
- Network function virtualization
- Programmable data plane

Application

- Network monitoring
- In-network computing
- Machine learning for networking

Learning objectives

Typical data structures and algorithms that are repetitively used in network system designs

How to quickly decide if an element exists in a large set?

How to efficiently count the occurrences for a large set of elements?

How to quickly find a match on the prefix of a string?

How to quickly decide if an element exists in a large set?

Use cases

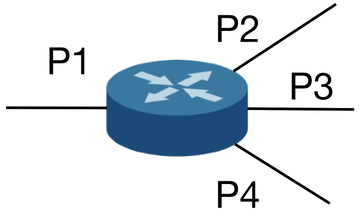
Networking tasks that involve membership detection

130.83.164.11
130.83.165.12
130.83.165.24
…

Access Control List (ACL)

Decides if an IP address is in the block list

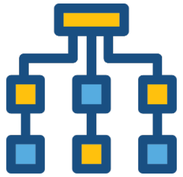
240.0.0.5 → {P1,P3}
240.0.0.6 → {P1,P2}
240.0.0.7 → {P2,P3}
240.0.0.8 → {P1,P2,P3}



IP Multicast

Decides if a router port should replicate a packet

10.0.2.10 → S1
10.0.3.10 → S2
10.0.4.10 → S3



Load Balancer

Decides if a source IP has been assigned to a server

Trivial solutions

Unordered list:

F	B	D	G	I	C	A	H

Linear search: O(n) time where n is the number of elements

Ordered list:

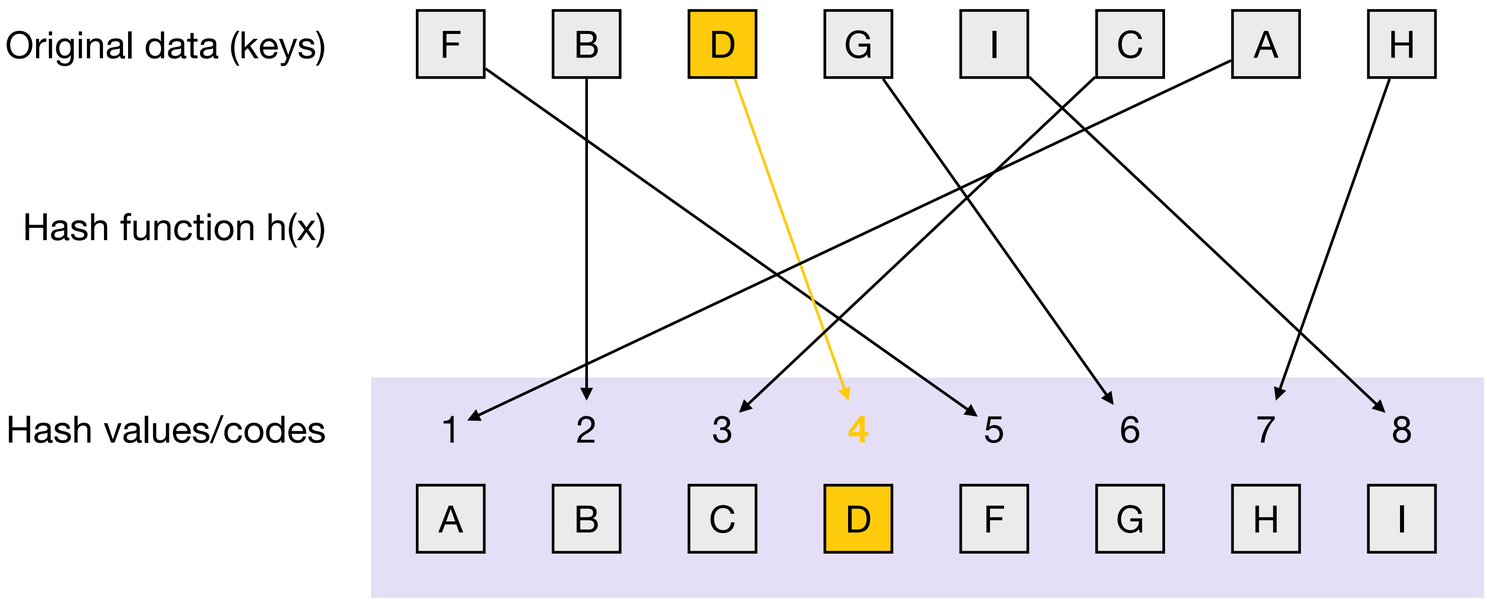
A	B	C	D	F	G	H	I

Binary search: O(logn) time where n is the number of elements

Can we achieve constant time O(1) search?

Hashing

Mapping data (of arbitrary size) to fixed-size values (indices here) with a function, sometimes also called scattered storage addressing



Hash table indexed by hash values

Properties of hashing

Advantages

- Constant time: O(1), only computing a function
- Can perform actions like add(x), find(x), and sometimes also delete(x)
- Simple, easy to implement

Drawbacks: what could be the problems of hashing?

- Does hashing provide us 100% correctness? What kinds of errors hashing may produce?
- How to handle these situations?
- What is the required storage space?

Properties of hashing

Advantages

- Constant time: O(1), only computing a function
- Can perform actions like add(x), find(x), and sometimes also delete(x)
- Simple, easy to implement

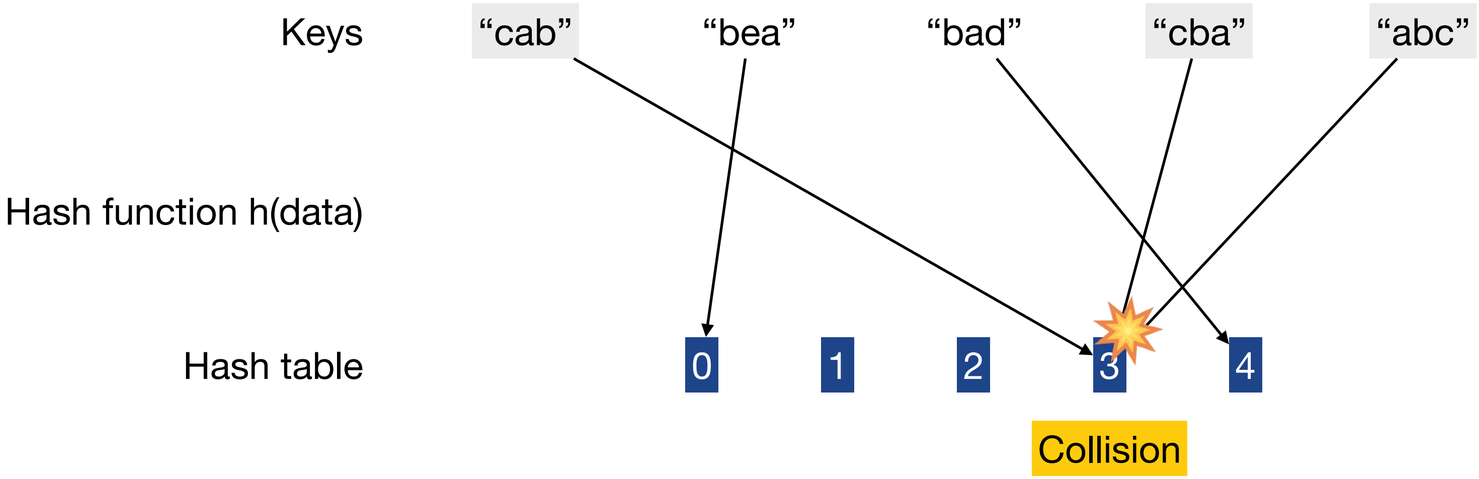
Drawbacks: what could be the problems of hashing?

- There can be **false positives** caused by hash “collisions”
- How to mitigate or handle hash collisions?
- Can we use even less space?

Hash collision

Describes the case where multiple data entries are mapped to the same hash value

Let a = 0, b = 1, c = 2, …
Hash function: h(data) = (∑ characters) mod table_size
table_size: size of the hash table



How can we mitigate or handle this issue?

Properties of good hash functions

Must return numbers: {0,…, table_size}

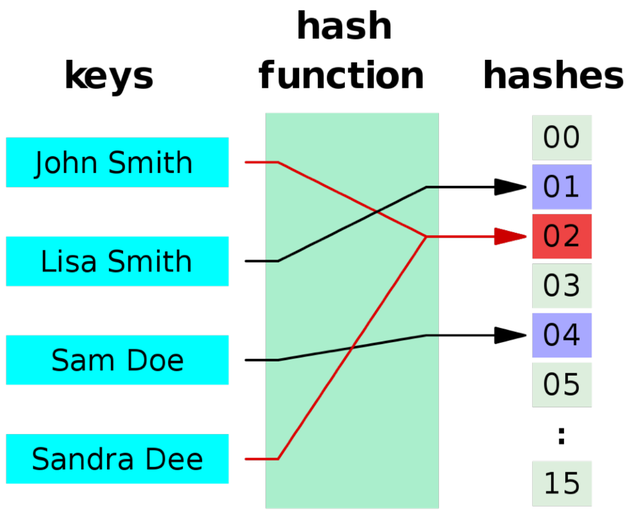
Must be deterministic: always returns the same value for the same key

Should be efficiently computable: O(1) time

Should not waste space unnecessarily:

- For every index, there is at least one key that hashes to it
- Load factor lambda = (# of keys) / table_size

Should minimize collisions: keys are nicely spread out



Modulo hashing

Performs hashing with a simple modulo operation

$$h(x) = x \% \text{table_size}$$

$$\downarrow$$

Use a **prime** number as table_size to achieve a better distribution of values

Do you know why we choose a prime number?

Modulo hashing

Performs hashing with a simple modulo operation

$$h(x) = x \% \text{table_size}$$

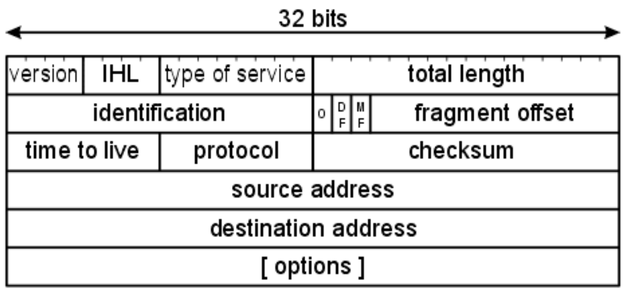
$$\downarrow$$

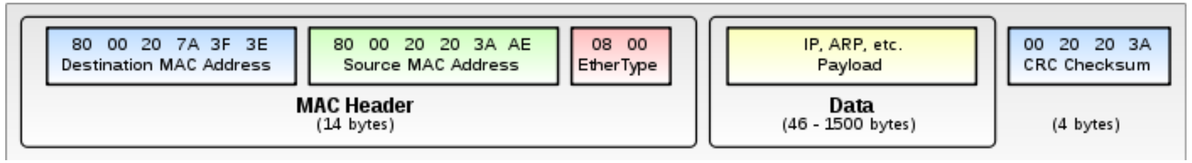
Use a **prime** number as table_size to achieve a better distribution of values

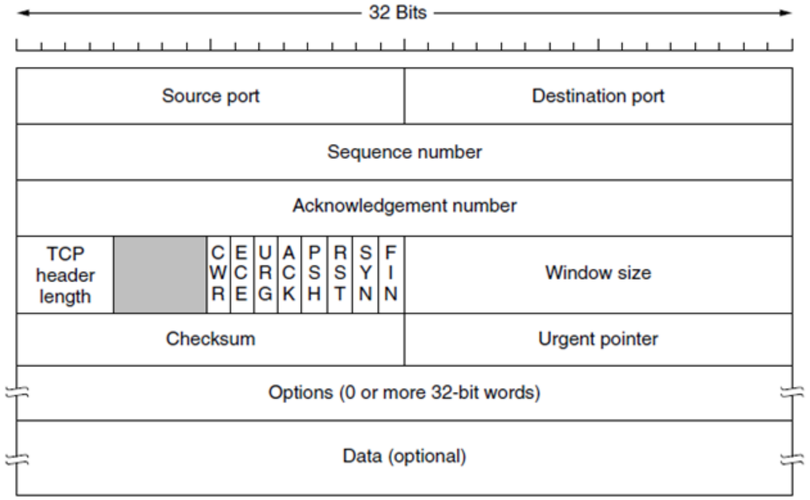
For uniformly distributed data, it does not matter. But when the data is non-uniformly distributed, data that shares common factors with table_size will be hashed to the same values. Prime numbers have the least number of factors.

Other hash functions

Widely used in network systems for fast lookups and integrity checks







Checksum, cyclic redundancy check, non-cryptographic hash functions (MurMur, FNV, Jenkins), cryptographic hash functions (MD5, SHA256, SHA512)

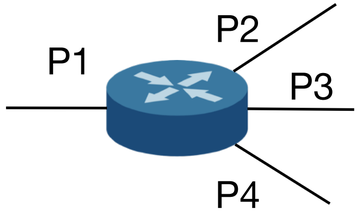
How to handle hash collisions?

Do we really care about hash collisions? If so, how to resolve the collision?

130.83.164.11
130.83.165.12
130.83.165.24
…

Access Control List (ACL)

240.0.0.5 → {P1,P3}
240.0.0.6 → {P1,P2}
240.0.0.7 → {P2,P3}
240.0.0.8 → {P1,P2,P3}



IP Multicast

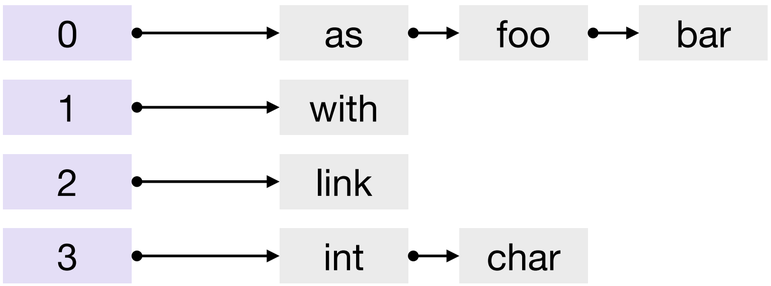
10.0.2.10 → S1
10.0.3.10 → S2
10.0.4.10 → S3

Load Balancer

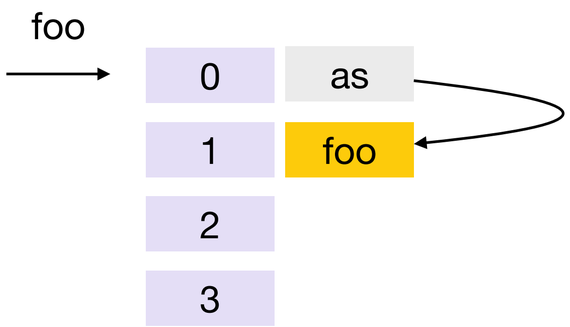
Can you tell if hash collisions are critically important in these networking scenarios?

Handling hash collisions

Designing a data structure that can resolve hash collisions



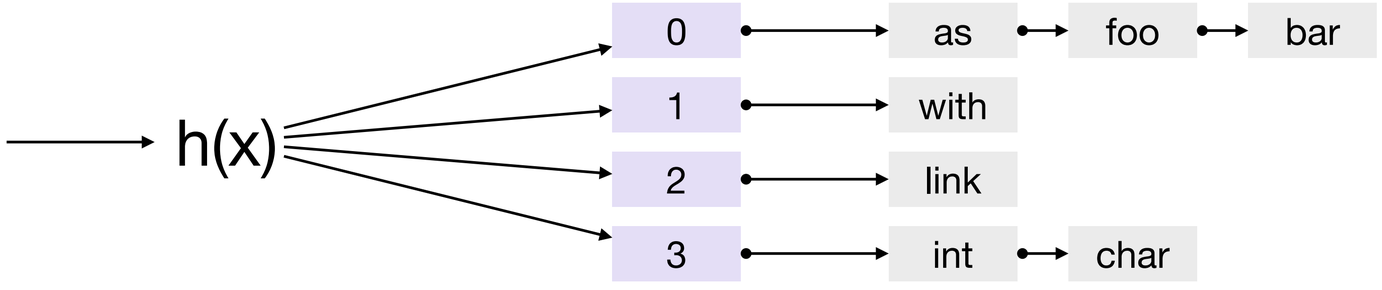
Separate chaining



Open addressing (linear/quadratic probing/cuckoo hashing)

Separate chaining

Creating a list of keys that map to the same hash value



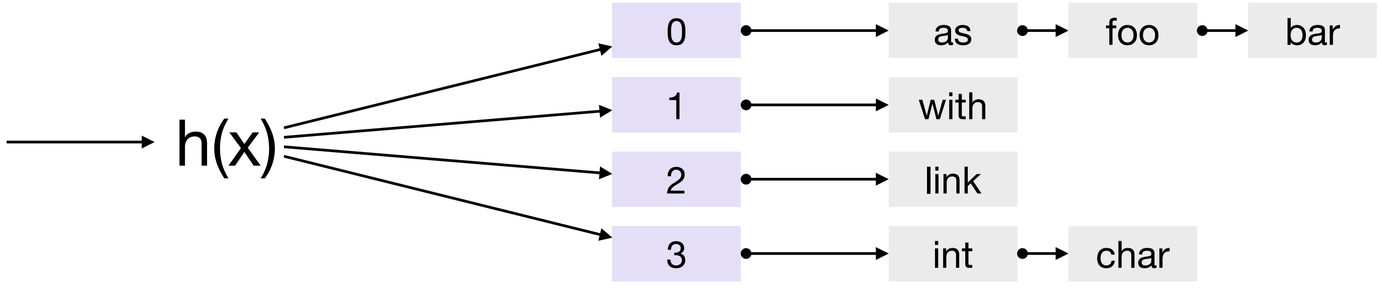
Still needs a good hash function to spread out the keys

A list of keys maintained in a linked list for each hash value

What are the consequences to the hashing performance?

Separate chaining

Creating a list of keys that map to the same hash value



Still needs a good hash function to spread out the keys

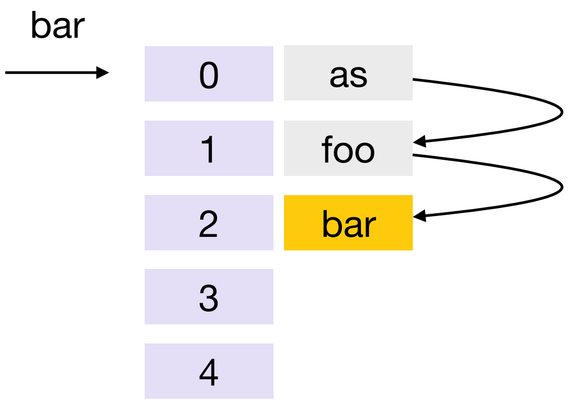
A list of keys maintained in a linked list for each hash value

Lookup time: average case O(N / table_size), worse case O(N)

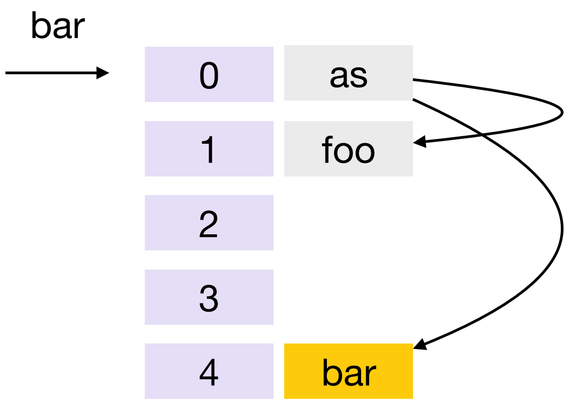
(N is the total number of keys)

Open addressing

Allowing to shift the values following a given strategy if collisions are detected



Linear probing (offset = 1, 2, 3,…)

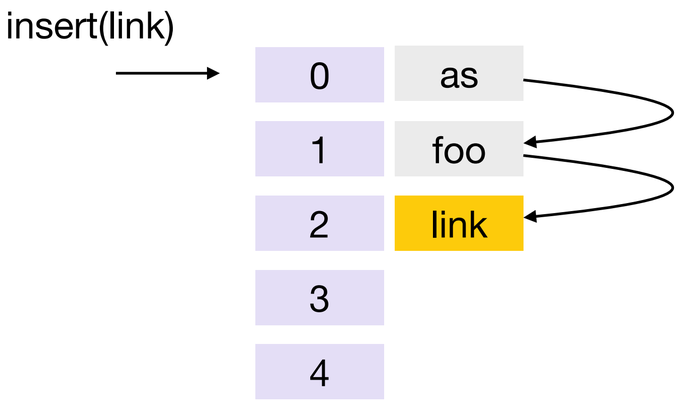


Quadratic probing (offset = 1, 4, 9,…)

Open addressing: linear probing

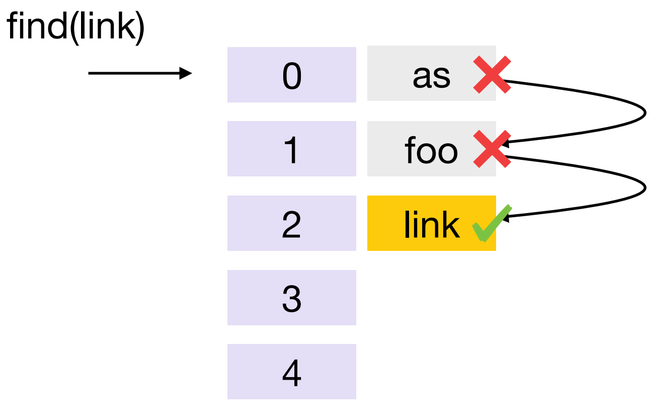
Probing with a linear offset: 1, 2, 3,…

Insertion



Upon collision, inset(x) finds the first slot after h(x) that is empty and inserts x in that slot

Lookup



Keep checking from h(x) until x is found in the hash table; does not exist if hitting an empty slot before x is found.

How to handle delete(x) operations?

Handling deletion operations in linear probing

delete(bar) →

0	as
1	foo
2	bar
3	int
4	char

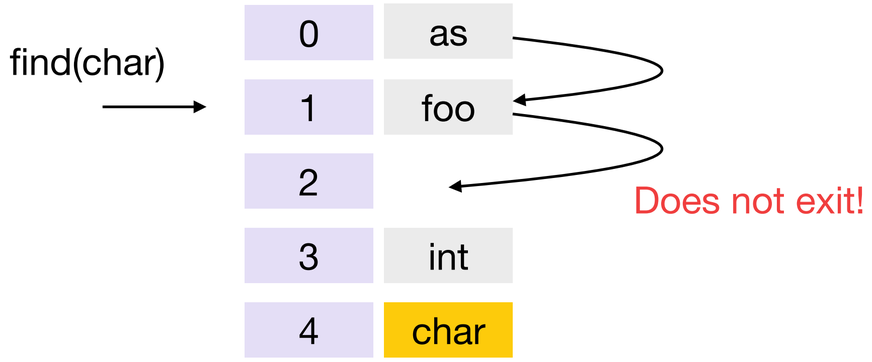
delete(bar) →

0	as
1	foo
2	
3	int
4	char

Is this correct?

Handling deletion operations in linear probing

Assume h(char) = 1



Problem: there are dependencies in locating the different keys in the hash table

Handling deletion operations in linear probing

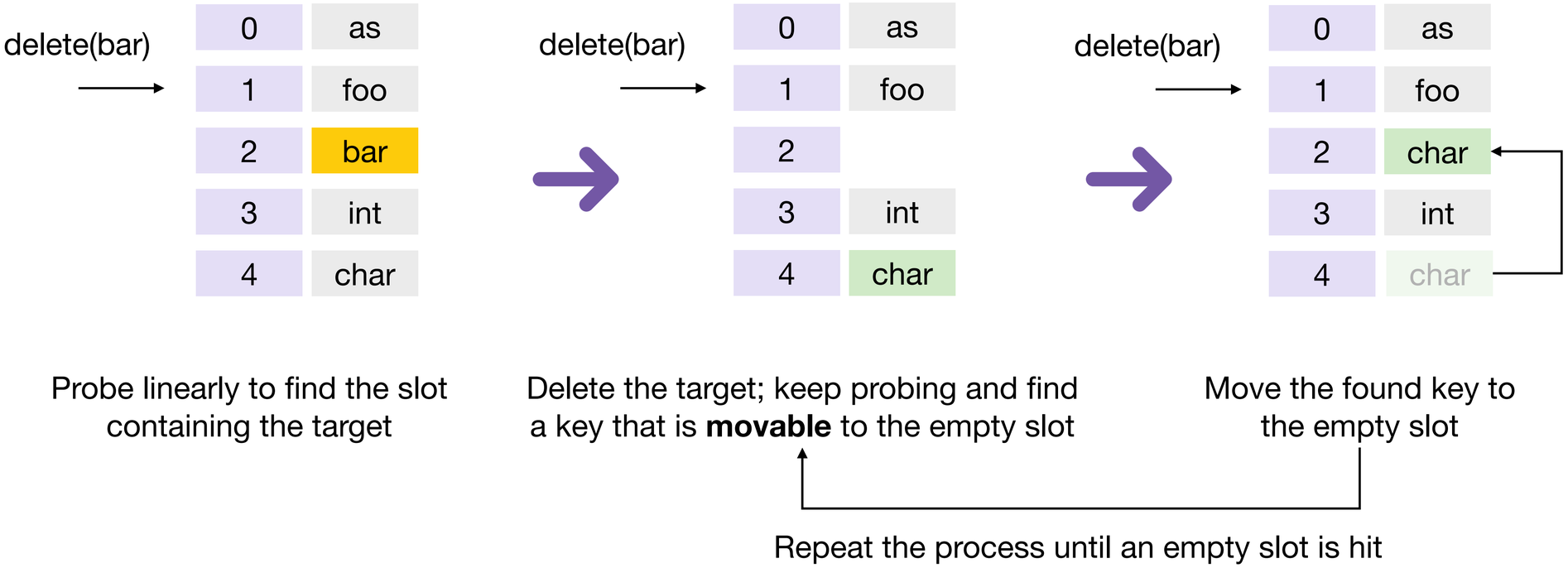
Assume h(char) = 1

find(char) →

0	as
1	foo
2	✕
3	int
4	char

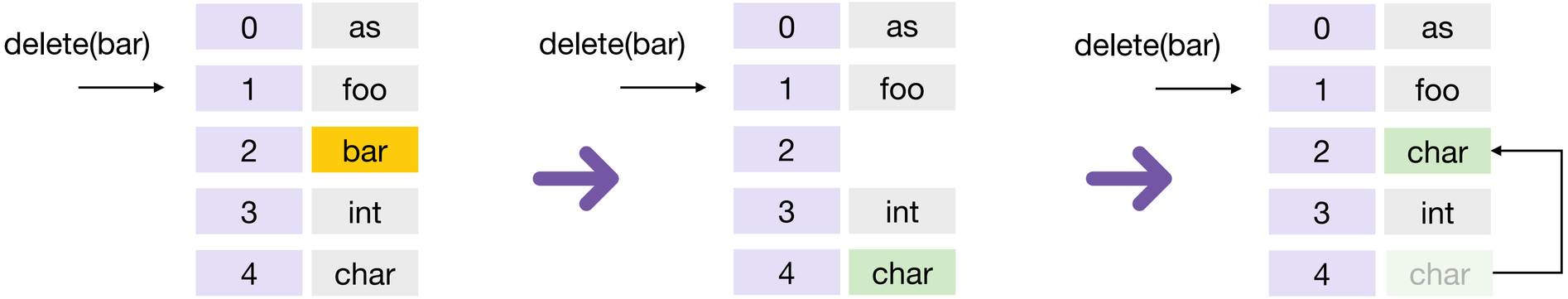
Maintain a flag of “deleted” for the emptied slots; adds in lookup time overhead

Handling deletion operations in linear probing



What defined a slot movable?

Handling deletion operations in linear probing



Probe linearly to find the slot containing the target

Delete the target; keep probing and find a key that is **movable** to the empty slot

Move the found key to the empty slot

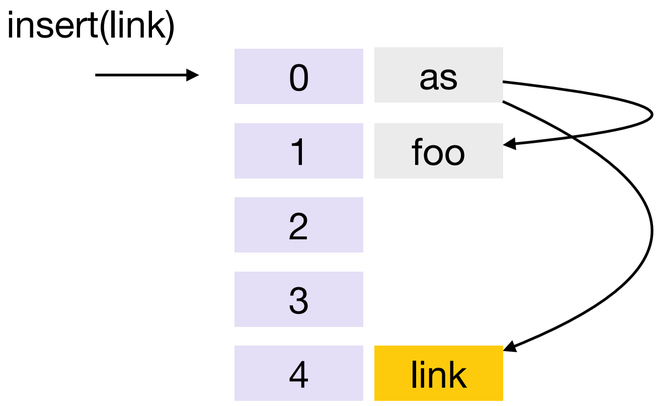
Repeat the process until an empty slot is hit

A slot is movable if the key contained in that slot has a hash value smaller than or equal to the hash value of the deletion target.

Open addressing: quadratic probing

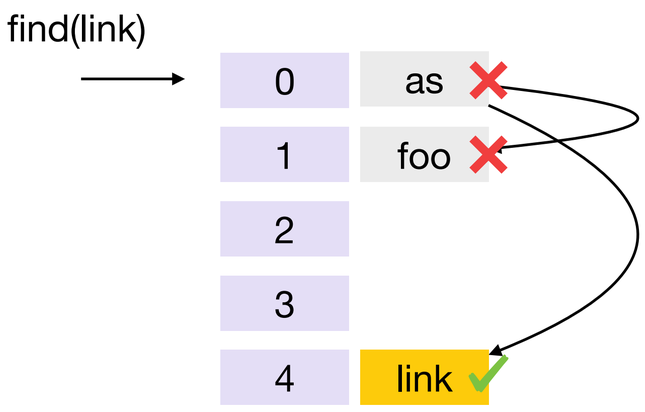
Probing with a quadratic offset: 1, 4, 9,…

Insertion



Upon collision, inset(x) finds the first slot after h(x) that is empty with a quadratic offset and inserts x in that slot

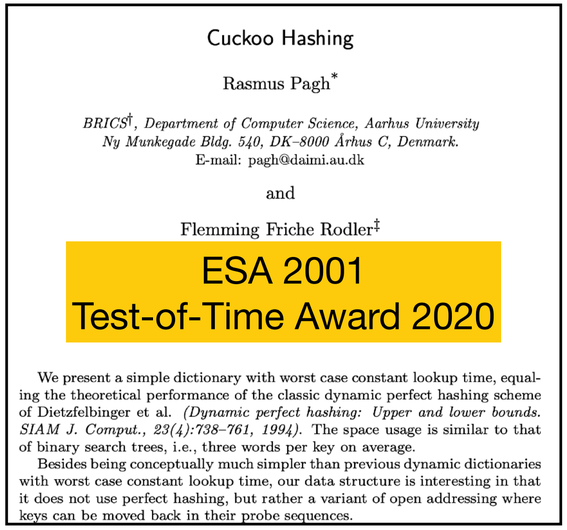
Lookup



Keep checking from h(x) with a quadratic offset until x is found in the hash table; does not exist if hitting an empty slot before x is found.

Open addressing: cuckoo hashing

Pushing other keys to a different location upon collisions



Cuckoo Hashing

Rasmus Pagh*

BRICS†, Department of Computer Science, Aarhus University
Ny Munkegade Bldg. 540, DK–8000 Århus C, Denmark.
E-mail: pagh@daimi.au.dk

and

Flemming Friche Rodler‡

ESA 2001
Test-of-Time Award 2020

We present a simple dictionary with worst case constant lookup time, equaling the theoretical performance of the classic dynamic perfect hashing scheme of Dietzfelbinger et al. *(Dynamic perfect hashing: Upper and lower bounds. SIAM J. Comput., 23(4):738–761, 1994).* The space usage is similar to that of binary search trees, i.e., three words per key on average.

Besides being conceptually much simpler than previous dynamic dictionaries with worst case constant lookup time, our data structure is interesting in that it does not use perfect hashing, but rather a variant of open addressing where keys can be moved back in their probe sequences.

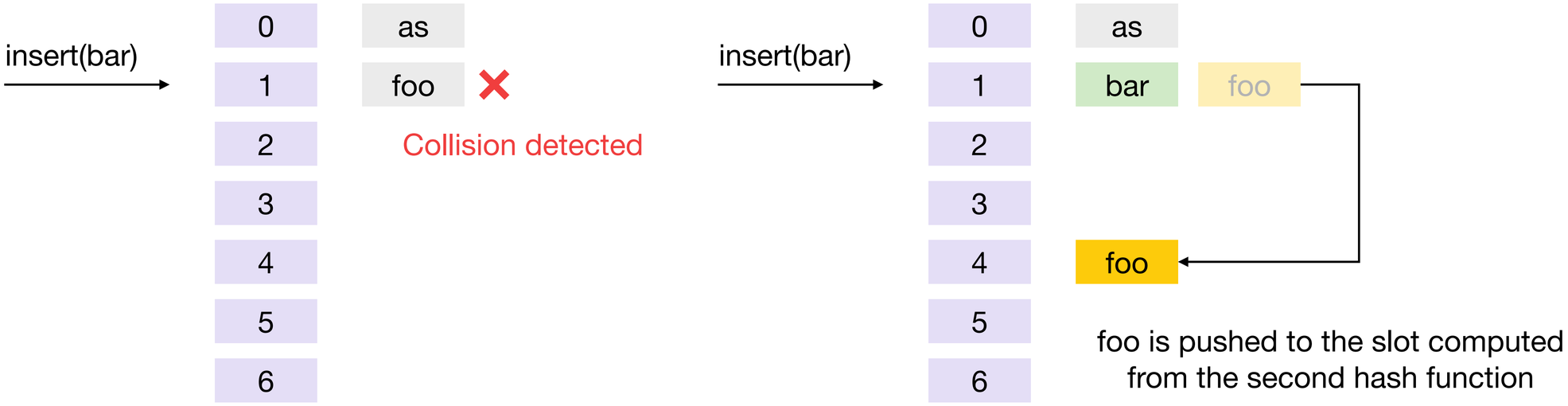
The name is derived from the behavior of some species of cuckoo, where the cuckoo chick pushes the other eggs or young out of the nest when it hatches.

Cuckoo hashing

Using two hash functions to generate two possible slots for each key

h1(foo) = 1, h2(foo) = 4

h1(bar) = 1, h2(bar) = 5

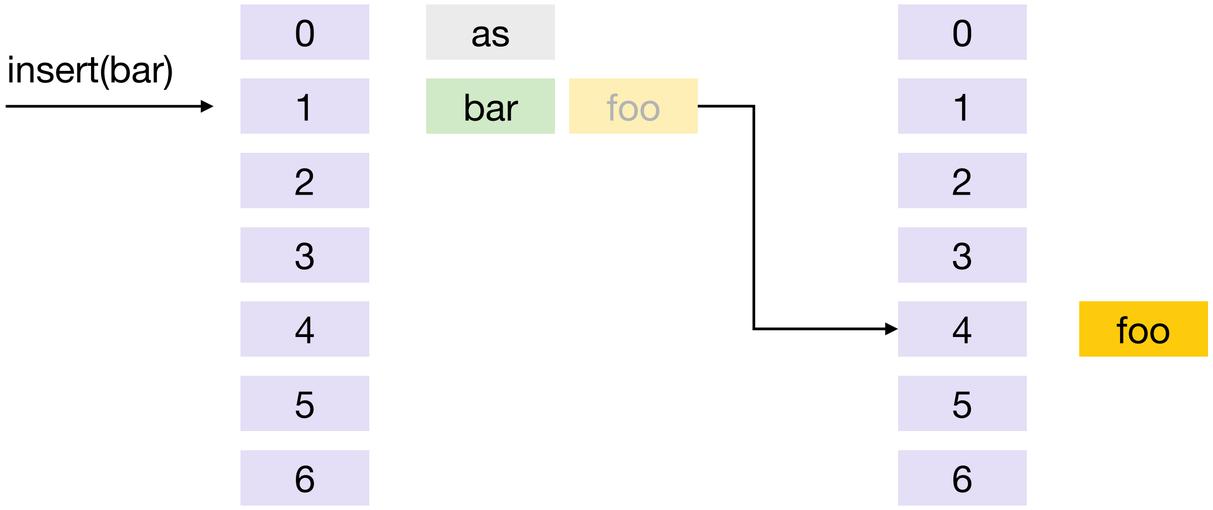


Cuckoo hashing implementation

Typical implementations use two separate hash tables, each indexed by one hash function

h1(foo) = 1, h2(foo) = 4 h1(as) = 0, h2(as) = 4

h1(bar) = 1, h2(bar) = 5

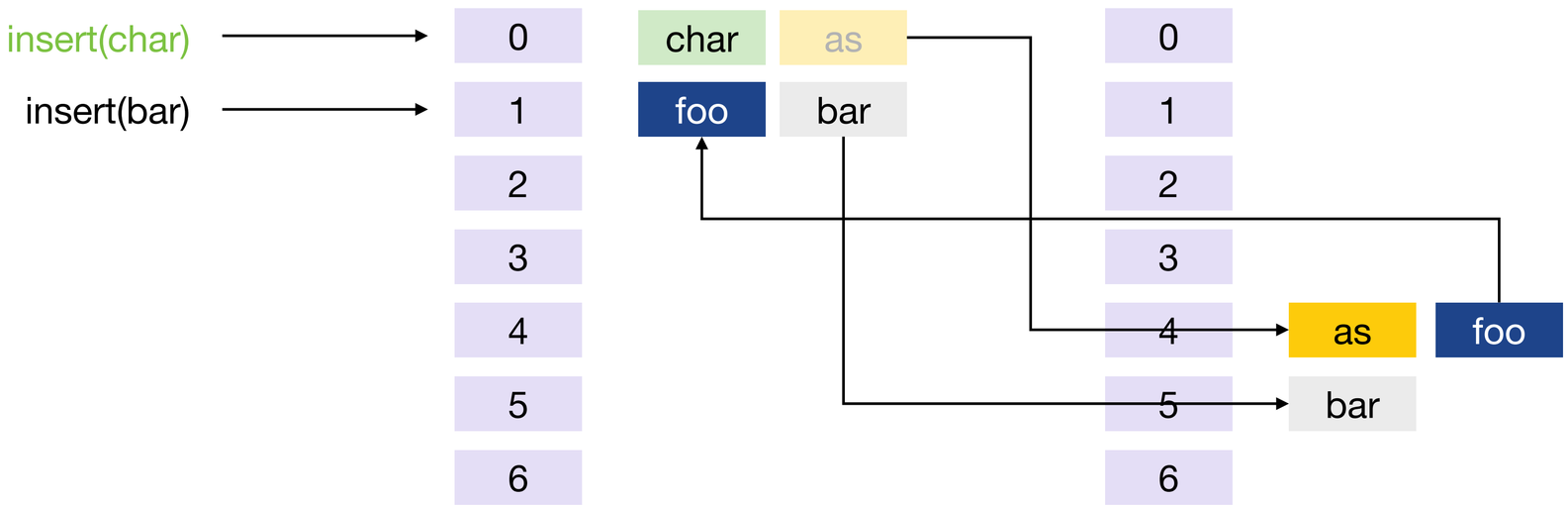


Cuckoo hashing operations

Insertion takes more time than lookup and deletion

h1(foo) = 1, h2(foo) = 4 h1(as) = 0, h2(as) = 4

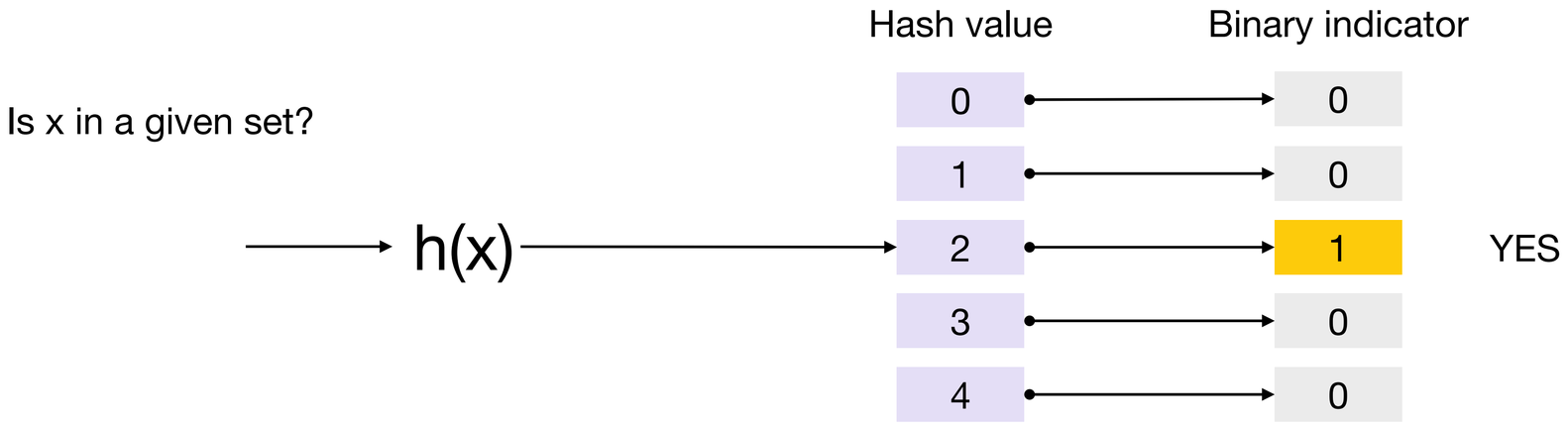
h1(bar) = 1, h2(bar) = 5 h1(char) = 0, h2(char) = 2



Insertion time worse case O(N), lookup time O(1), deletion time O(1)

Membership determination with hashing

Assume we do not have enough space to store all the keys, but
we want to answer membership determination queries



Set the binary indicator to 1 at insertion; return true if the binary indicator is 1 at lookup.

False positive rate analysis

Assume we have in total N keys and we use a hash table of M slots

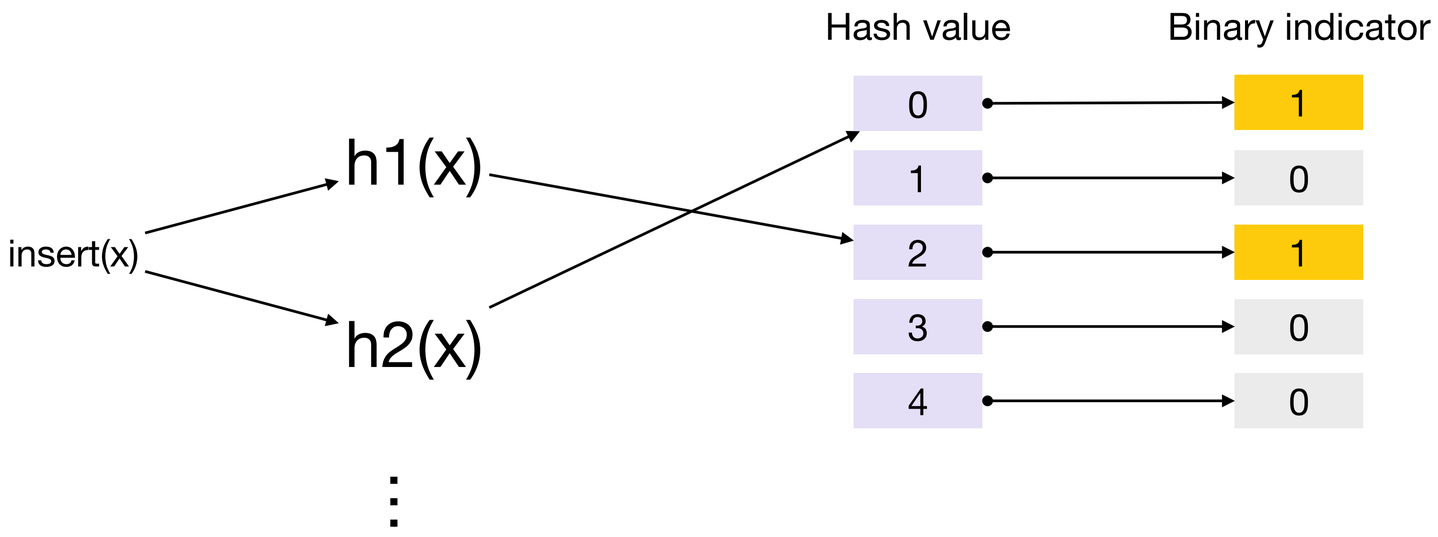
Probability of a key mapped into a particular slot: 1/M → Probability of a key **not** mapped into a particular slot: 1-1/M → Probability of a slot to be 0 after inserting N keys: (1-1/M)^N → False positive rate (FPR): 1-(1-1/M)^N

# of keys	# of slots	FPR
1000	10,000	9.5%
1000	100,000	1%

Roughly 100x number of slots is required to have an FPR lower than 1%.

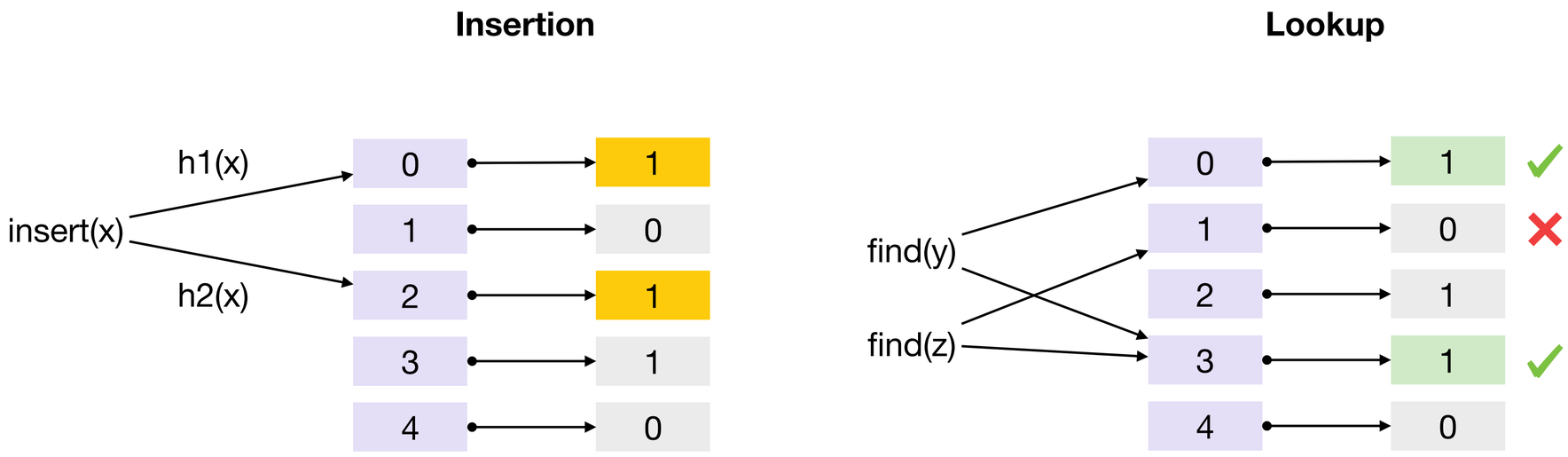
Bloom filter

Typically using multiple hash functions to lower collision rate



Bloom filter: insertion and lookup

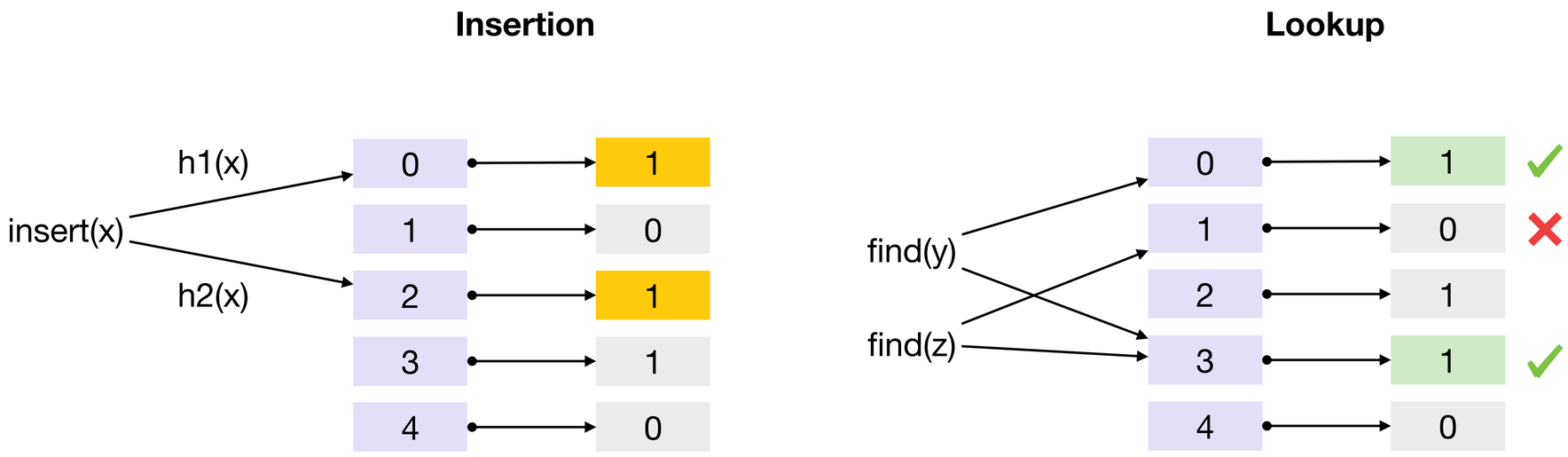
Setting the binary indicators corresponding to the hash values from the input to 1 if 0



Can we delete a key from the Bloom filter?

Bloom filter: insertion and lookup

Setting the binary indicators corresponding to the hash values from the input to 1 if 0



A basic Bloom filter does not support deletion since the indicators may be shared by other keys.

False positive rate analysis

Assume we have in total N keys and we use a Bloom filter of M slots with K hash functions

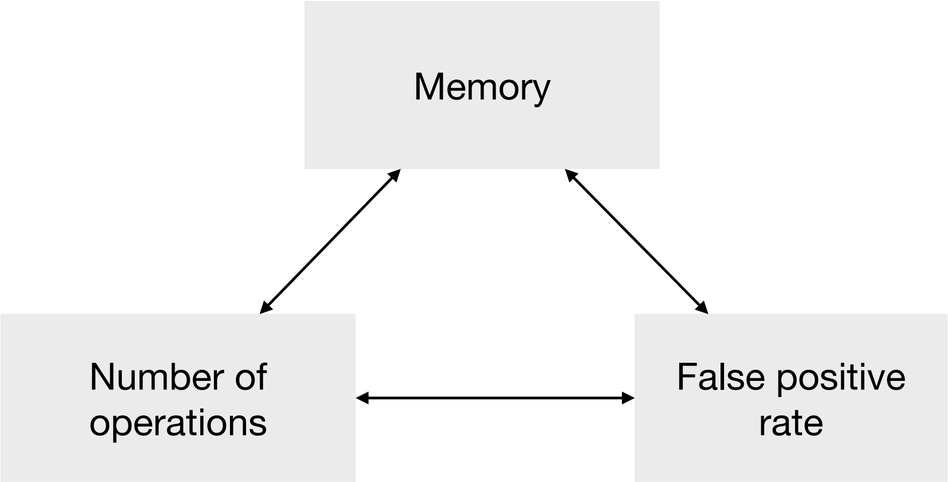
Probability of a key mapped into a particular slot: $1/M$ → Probability of a key **not** mapped into a particular slot: $1-1/M$ → Probability of a slot to be 0 after inserting N keys each with K hashes: $(1-1/M)^{KN}$ → False positive rate (FPR): $(1-(1-1/M)^{KN})^{K}$

# of keys	# of slots	# of hash functions	FPR
1000	10,000	7	0.82%
1000	100,000	7	≈0%

Consumes almost 10x less space than the single-hash case, but requires slightly more computation for the operations.

Tradeoffs in Bloom filter design

How to dimension your Bloom filters to achieve the lowest FRP with the least overhead?



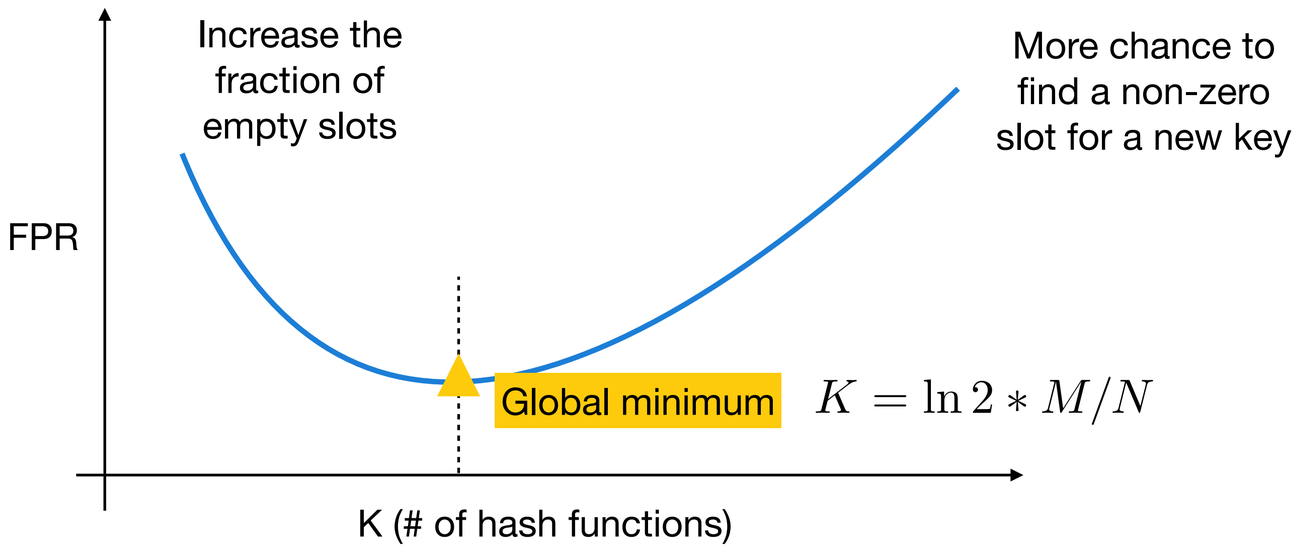
How to decide the number of slots and the number of hash functions to satisfy a given FPR?

Dimensioning Bloom filters

N keys, M slots, K hash functions, and FPR false positive rate

$$FPR = \left(1 - \left(1 - \frac{1}{M}\right)^{KN}\right)^{K} \approx \left(1 - e^{-KN/M}\right)^{K}$$

Asymptotic approximation

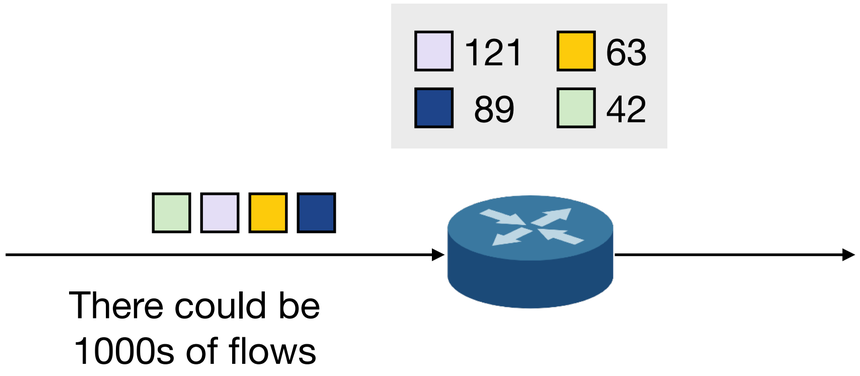


How to efficiently count the occurrences for a large set of elements?

Example: heavy hitter detection

Detecting the top-K flows (in terms of traffic volume, #packets) that have passed through a given router

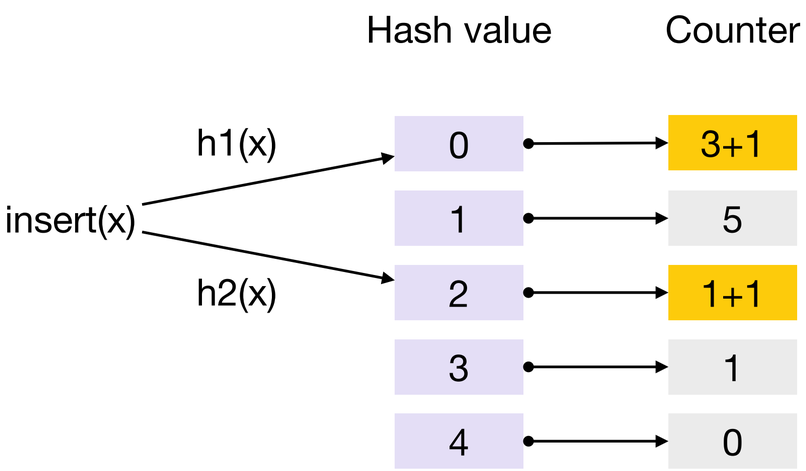
A flow is defined by a 5-tuple: <src_ip, dst_ip, src_port, dst_port, protocol>



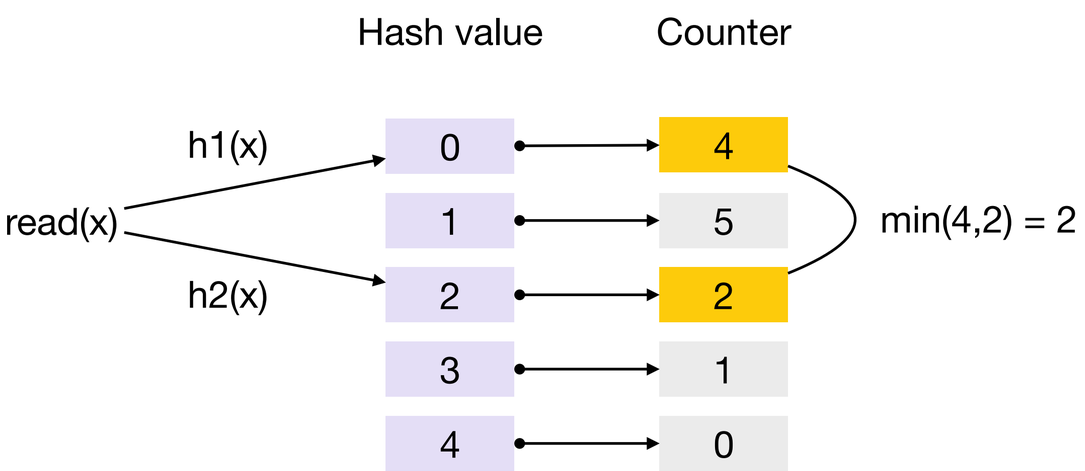
Routers are resource-limited, so creating **counters** for each separate flow is not scalable.

Counting Bloom filter

Extension to Bloom filter that can count the occurrences of keys



Increment the counters corresponding to the hash values

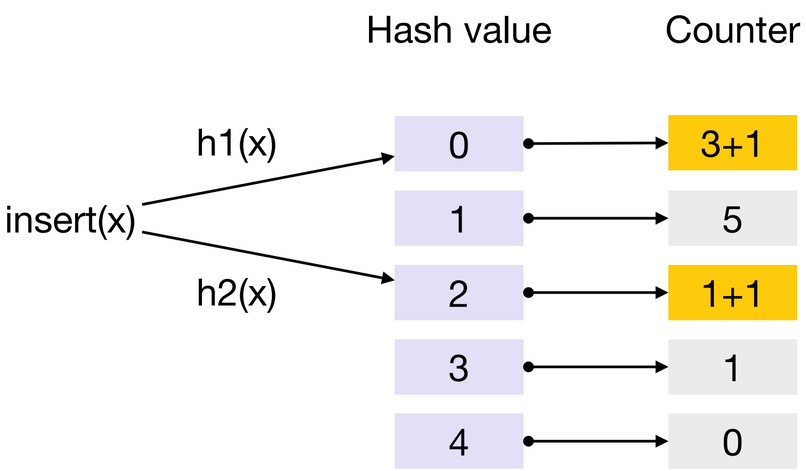


Lookup the counters corresponding to the hash values with the minimum count

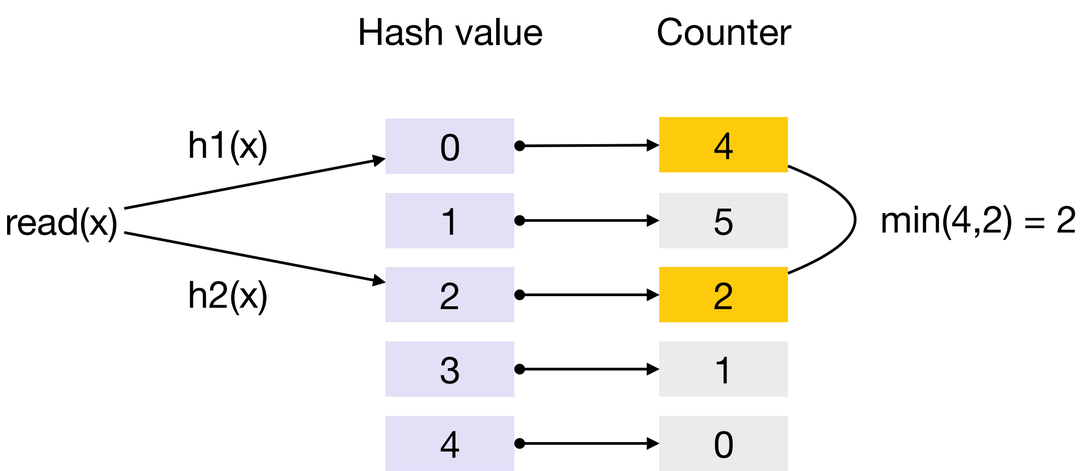
Is the count always correct? If not, what guarantees do we have?

Counting Bloom filter

Extension to Bloom filter that can count the occurrences of keys



Increment the counters corresponding to the hash values

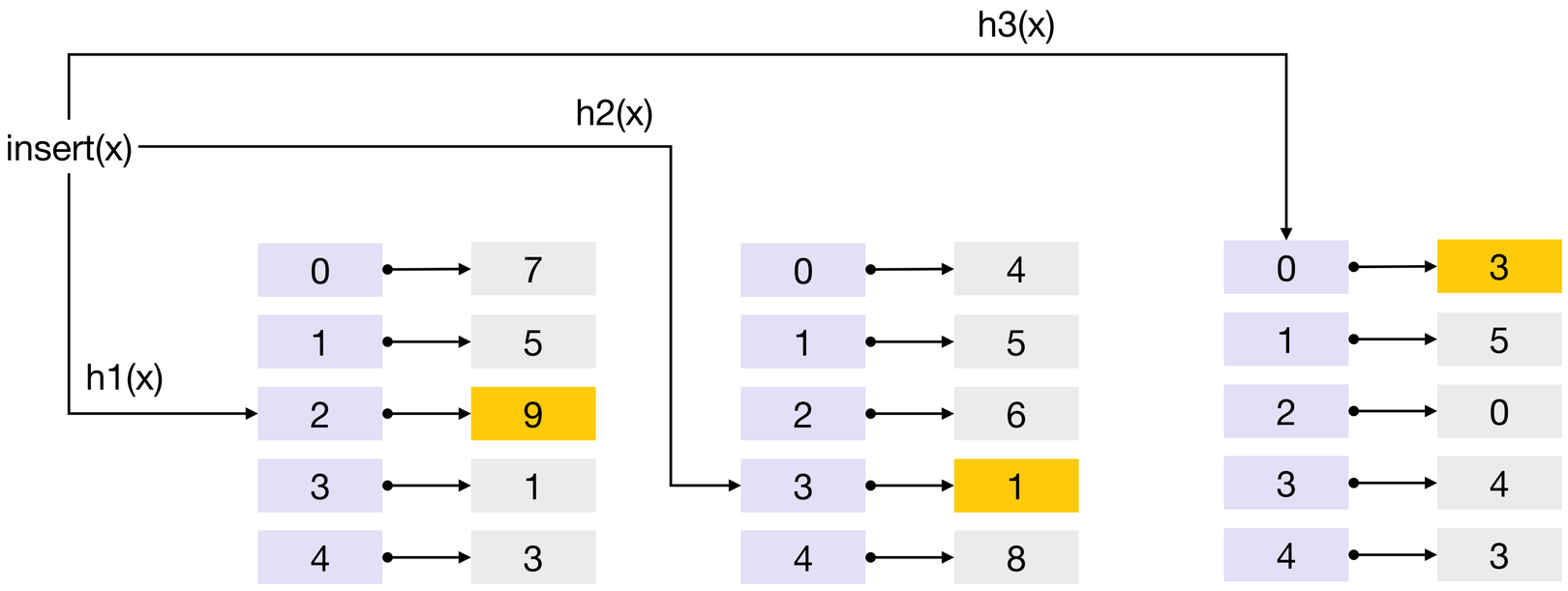


Lookup the counters corresponding to the hash values with the minimum count

A counting Bloom filter **cannot** ensure correctness; the count is an **upper-bound** of the actual count.

Count-min sketch

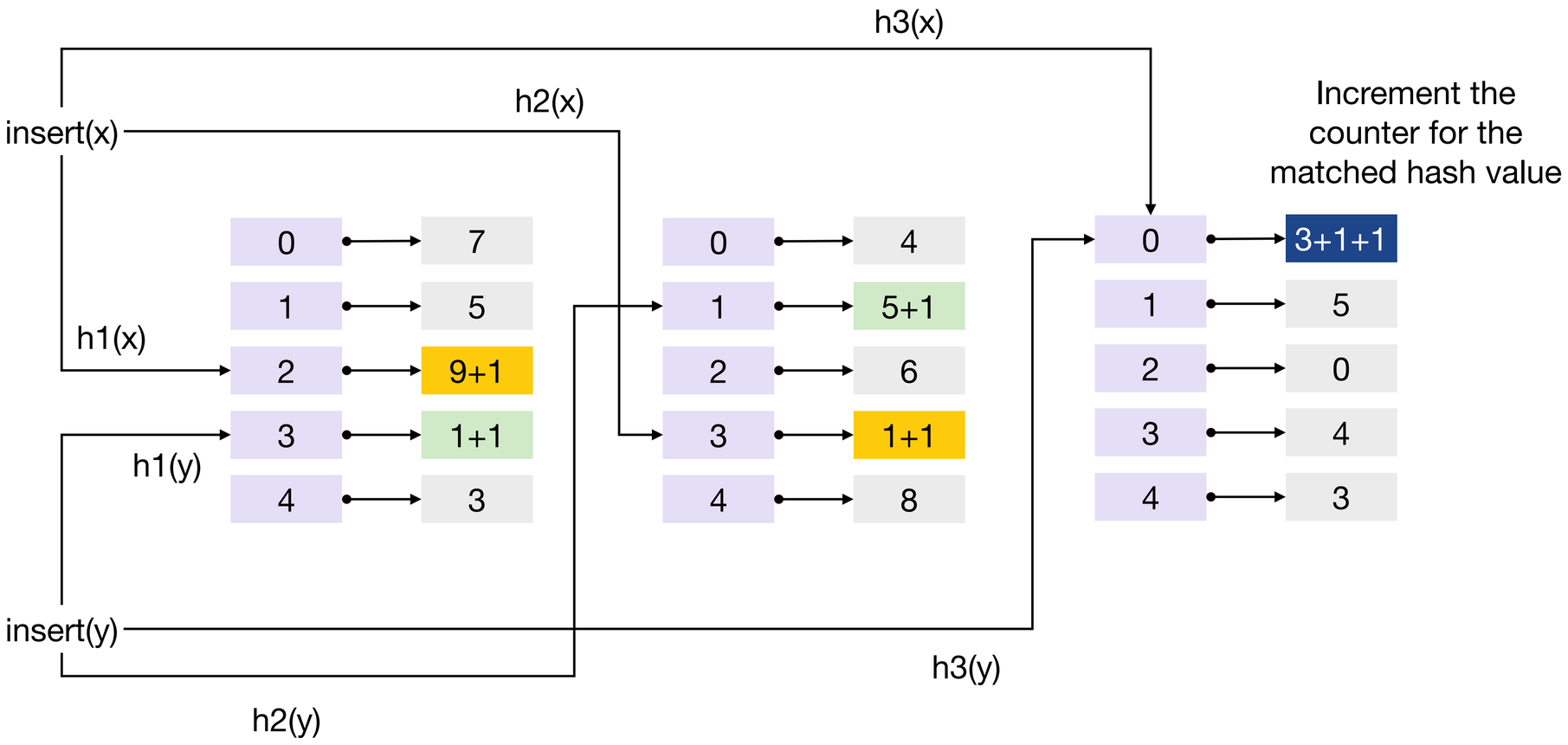
A slight improvement to the counting Bloom filter



Three hash functions are performed, each mapped to an array of counters (hash tables).

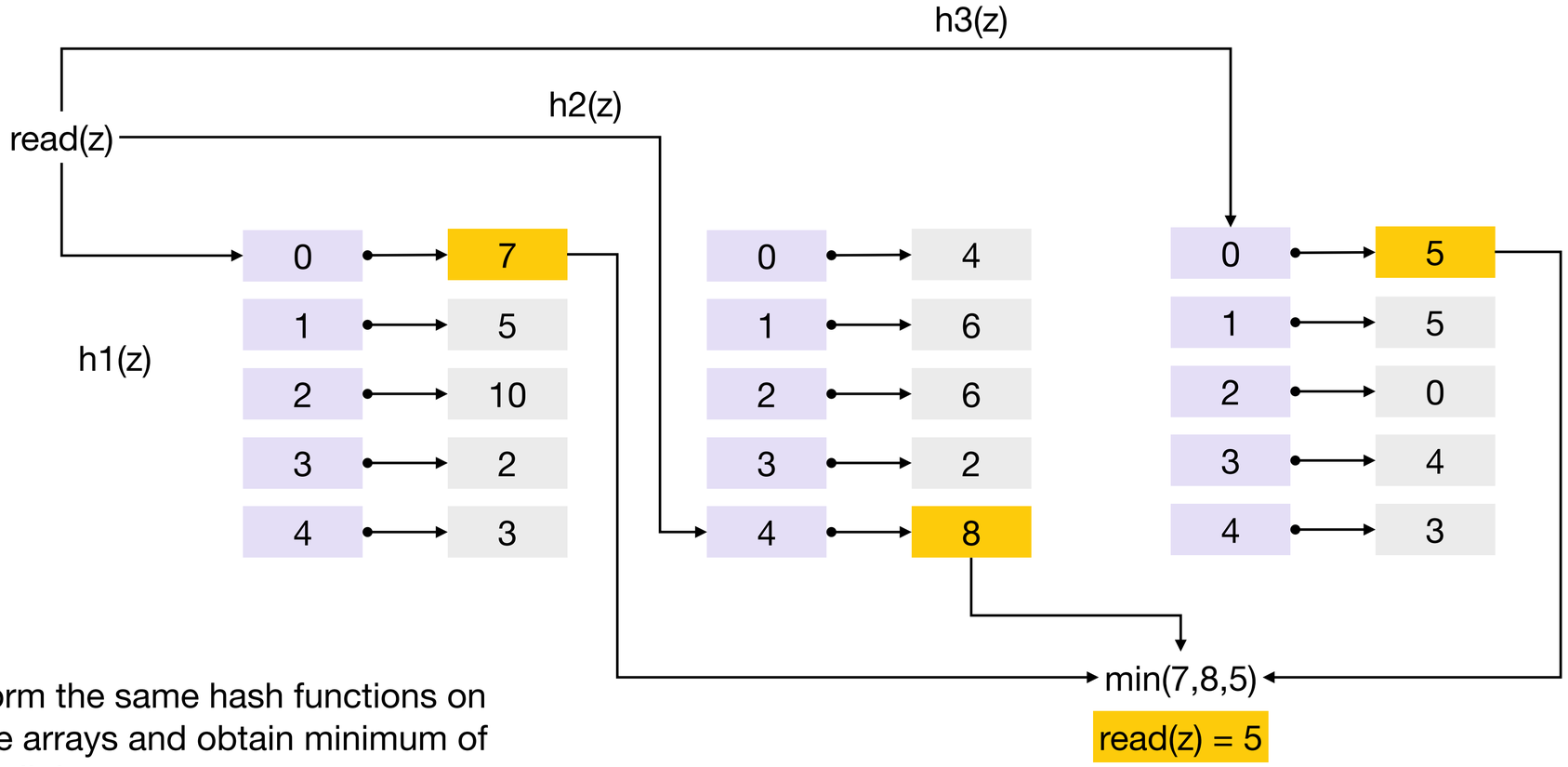
Count-min sketch insertion

Incrementing the counters for the computed hash values



Count-min sketch insertion

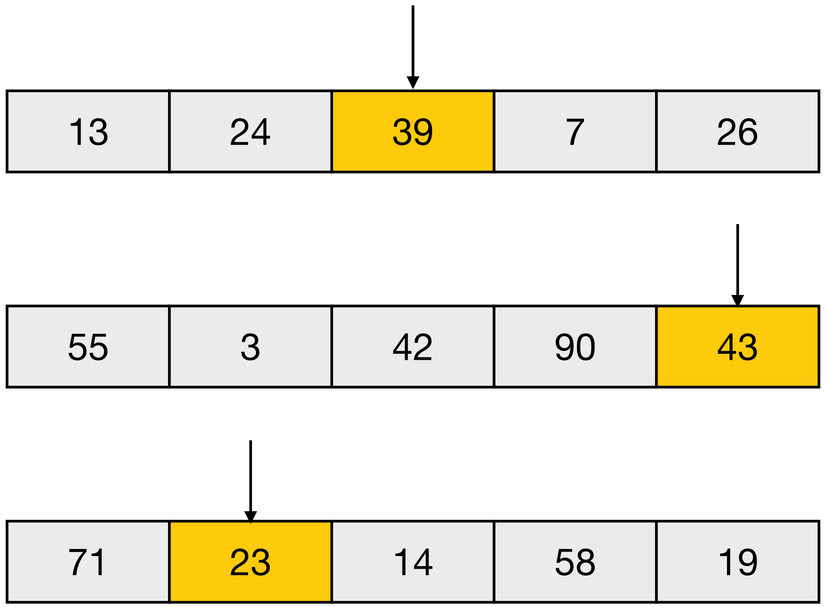
How to read the count from the count-min sketch?



Perform the same hash functions on all the arrays and obtain minimum of all the counters as output.

Exercise

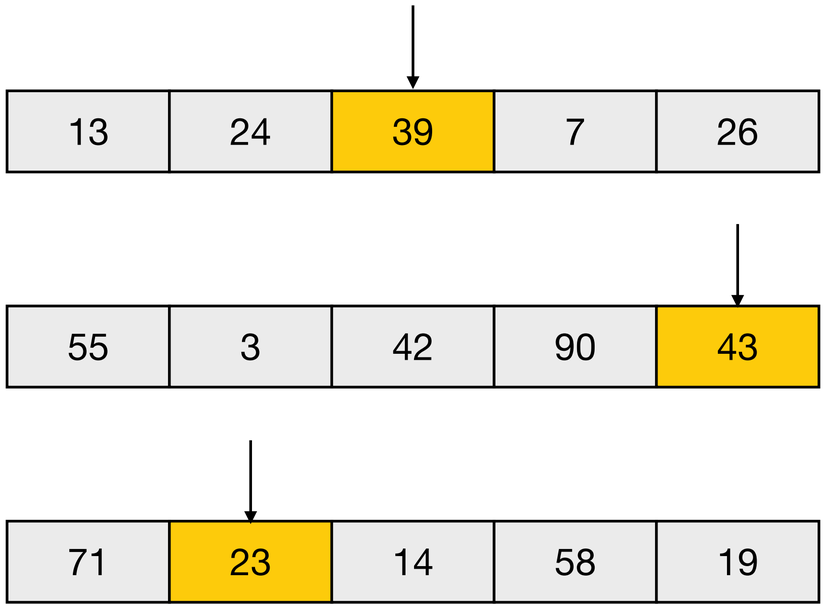
What is the count of the following read operation on a count-min sketch?



Is the count always correct? If not, what guarantees do we have?

Exercise

What is the count of the following read operation on a count-min sketch?



A count-min sketch **cannot** ensure correctness; the count is an **upper-bound** of the actual count.

Count-min sketch performance

A count-min sketch uses the same principles as a counting Bloom filter, but is designed to have **provable L1 error bounds** for frequency queries

$$Pr[\hat{x}_i - x_i \geq \varepsilon ||x||_1] \leq \delta$$

Estimated frequency

True frequency

Sum of frequencies

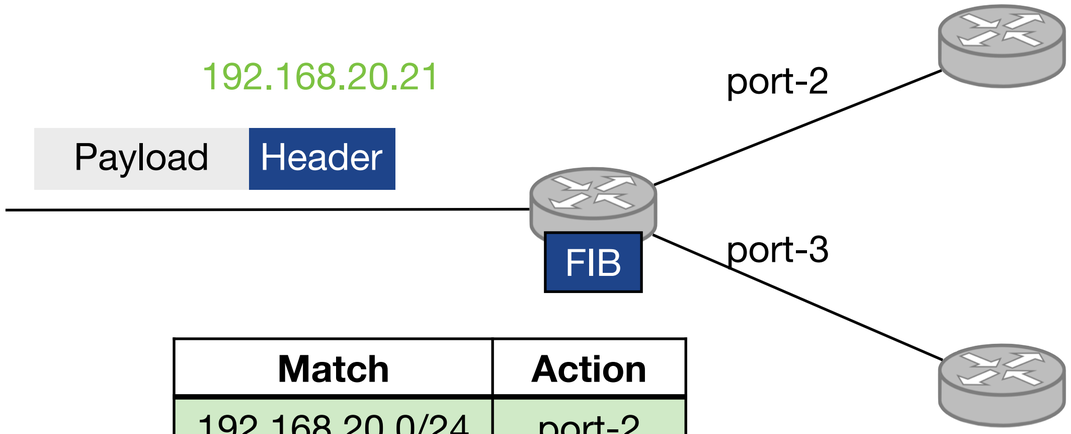
Favoring high-frequency elements: error is lower for high-frequency elements, but much higher for low-frequency elements

http://theory.stanford.edu/~tim/s15/l/l2.pdf

How to quickly find a match on the prefix of a string?

Example: IP lookup

Match on IP prefix and take an action: packet filtering, policy routing, rate limiting or traffic shaping



Match	Action
192.168.20.0/24	port-2
192.168.0.0/16	port-1
139.70.8.0/24	drop

Can be performed in **software** (in operating systems) and **hardware** (routers/firewalls)

Software implementation

How to make the implementation efficient?

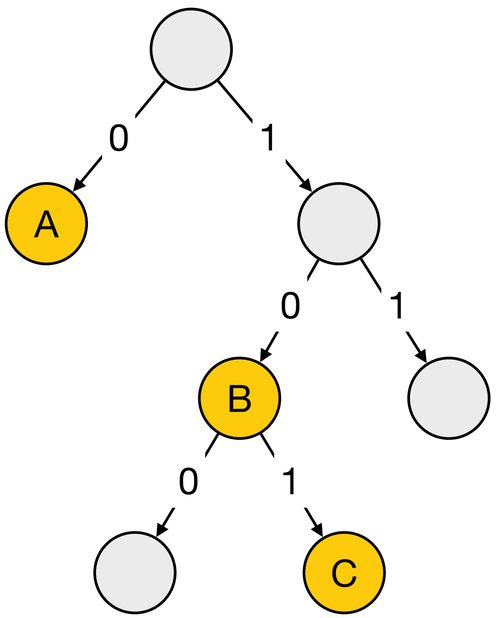
Target to match	192.168.20.21	**11000000.10101000.00010100**.00010101
Entries to match	192.168.20.0/24	**11000000.10101000.00010100**.00000000
	192.168.0.0/16	**11000000.10101000**.00000000.00000000
	139.70.8.0/24	**10001011.01000110.00001000**.00000000

We can do **linear search**, but it too time-consuming if we have a long list of entries to match.

IP lookup with Trie

Trie is a data structure whose nodes have a part of the prefix.
The name is derived from the word re**trie**val.

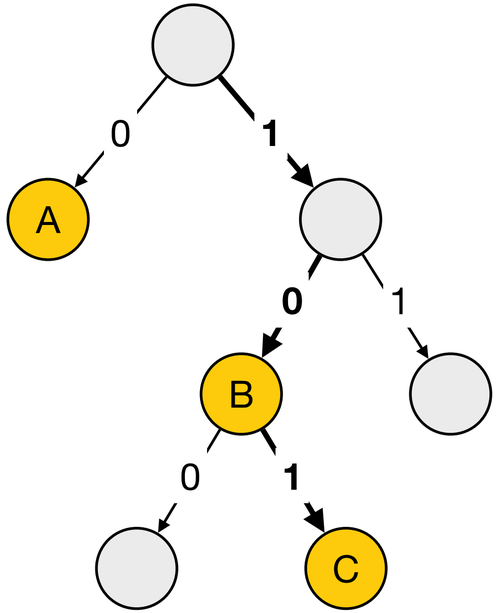
Prefix	Output-port
0*	A
10*	B
101*	C



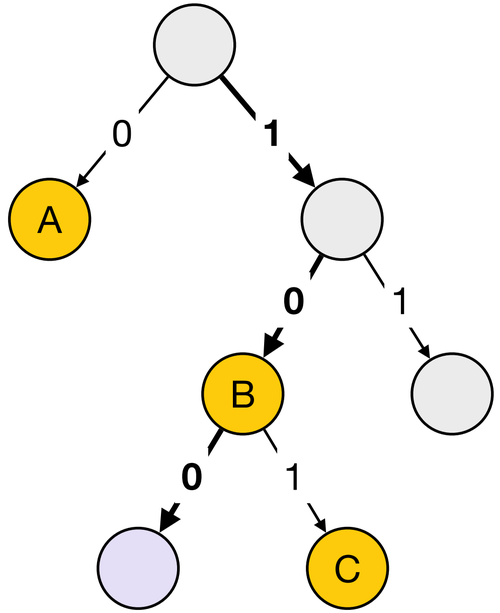
Trie lookup examples

Prefix	Output-port
0*	A
10*	B
101*	C

Input: 1011 → matched to router C



Input: 1000 → matched to router B



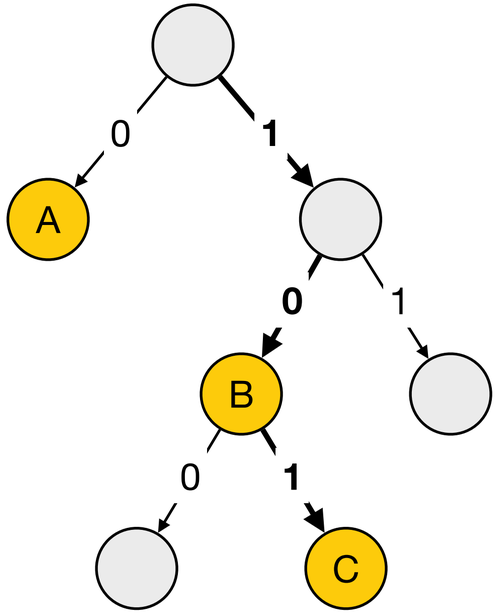
Longest prefix matched was 10

How many memory accesses do we need for an IP address lookup?

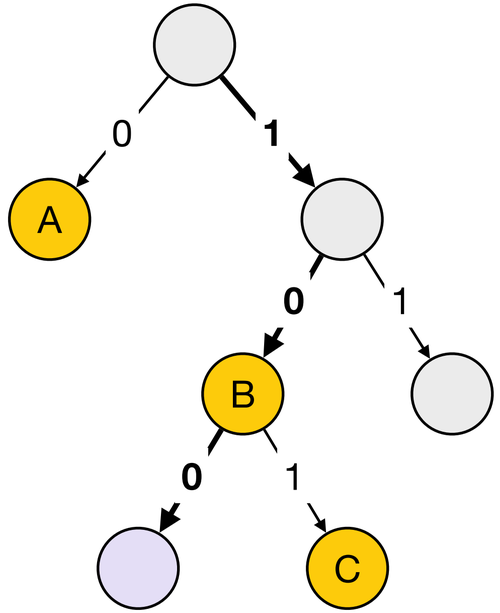
Trie lookup examples

Prefix	Output-port
0*	A
10*	B
101*	C

Input: 1011 → matched to router C



Input: 1000 → matched to router B



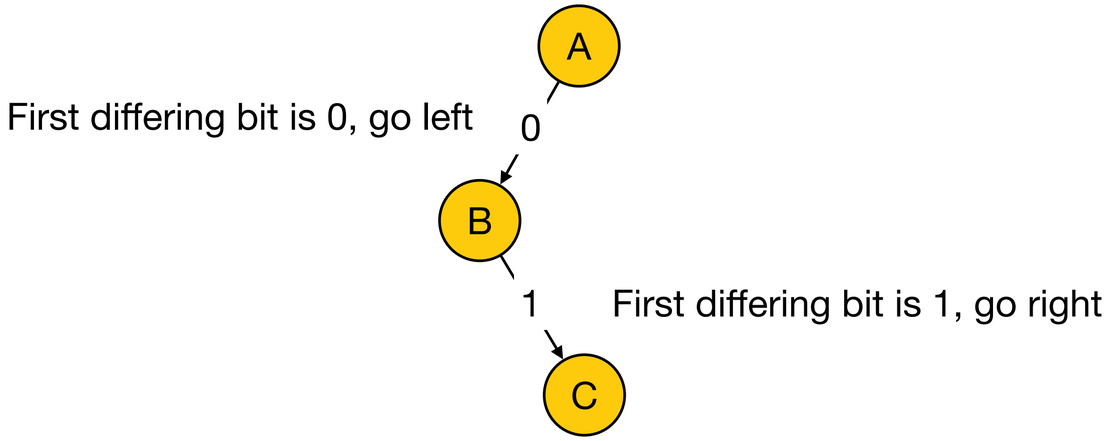
Longest prefix matched was 10

An IP address has 32 bits and we need to access the memory on every bit in the worst case.

PATRICIA trie

A special case of the radix trie where the radix is 2

192.168.0.0/16	**11000000.10101000**.00000000.00000000	A
192.168.20.0/24	11000000.10101000.00010100.00000000	B
192.168.20.127/28	11000000.10101000.00010100.11110000	C



PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric

DONALD R. MORRISON

Sandia Laboratory, Albuquerque, New Mexico*

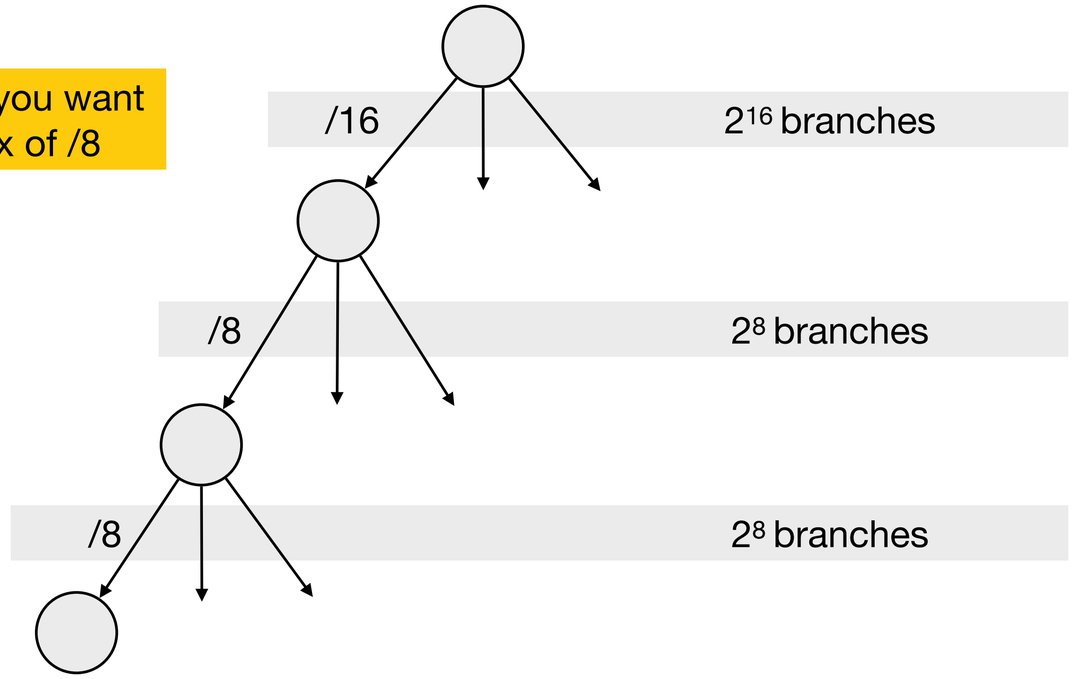
JACM 1968

Still needs 32 memory accesses in the worst case

Direct trie

Saving memory accesses by combining segments in entries

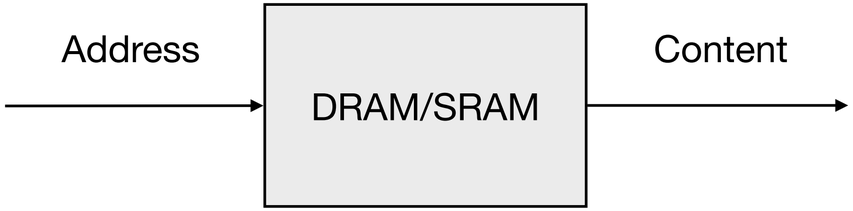
2^8 repeated entries if you want to match on a prefix of /8



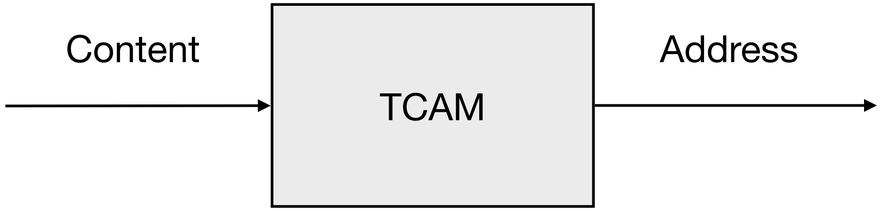
By combining segments, the number of memory access is reduced to 3, but the memory usage is inefficient.

Hardware implementations

Ternary content addressable memories (TCAM)



Given an address, retrieves the content at the address

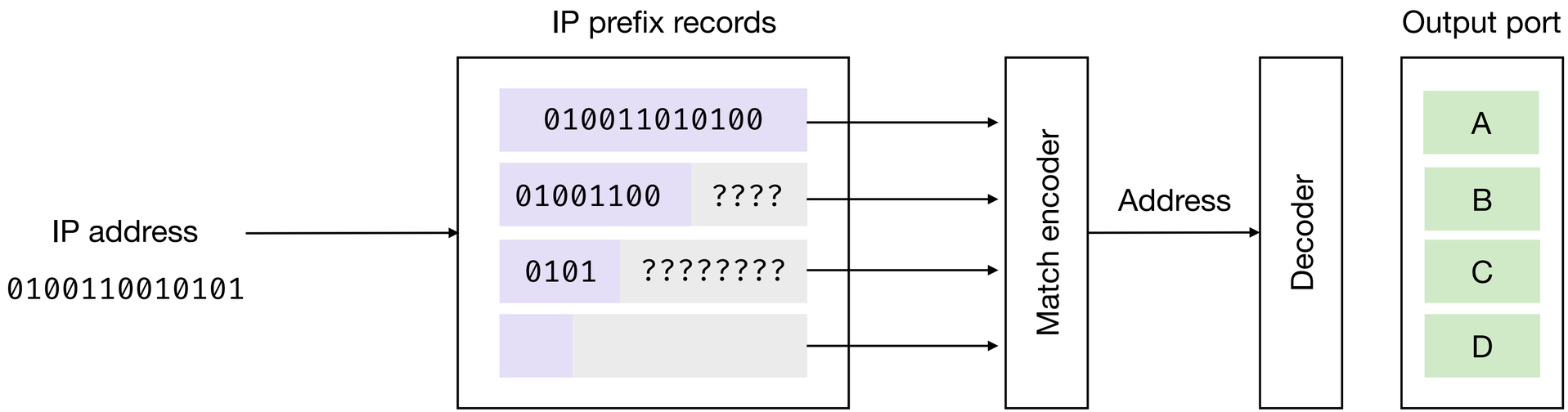


Given content, retrieves the address of the content

TCAM

A hardware device that supports to match on a set of records in constant time (one iteration)

- CAM supports only two states (0/1) in each bit position: widely used in switches for MAC address matching
- TCAM extends CAM by allowing for 3 states (0/1/?) in each position: useful for IP prefix matching
- Disadvantages: expensive, power-consuming



Summary

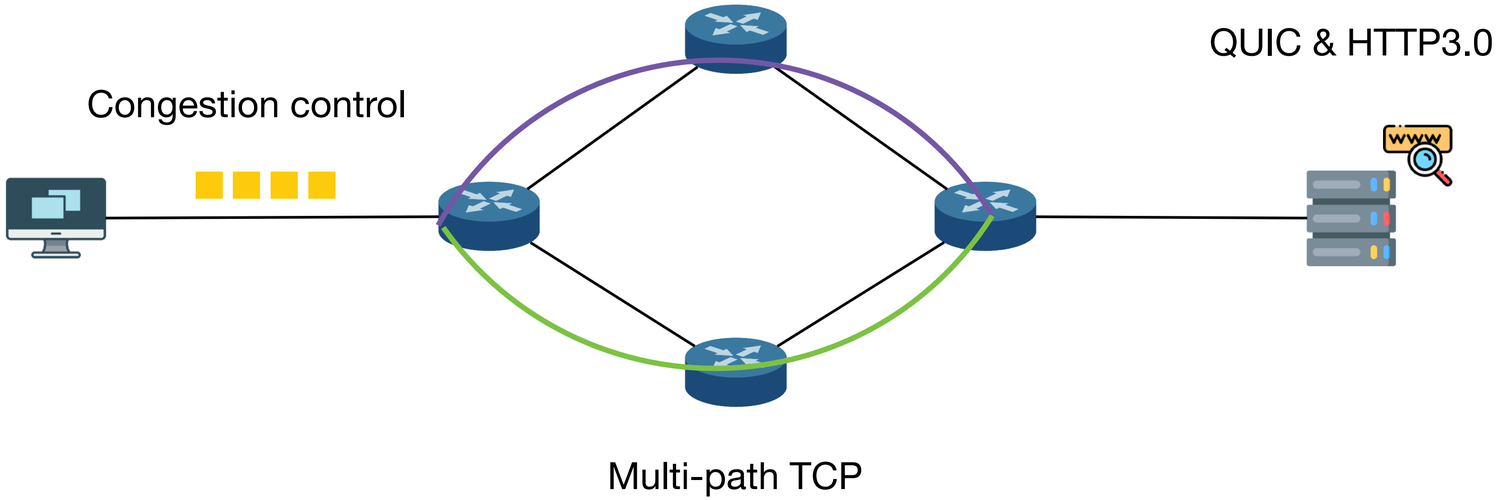
How to quickly decide if an element exists in a large set?

Hashing, Bloom filter

How to efficiently count the occurrences for a large set of elements?

Counting Bloom filter, count-min sketch

How to quickly find a match on the prefix of a string?

Trie-based prefix matching, TCAM

Next lecture: network transport